Midterm Project Report

Team 2: Wenjing (Wendy) Zheng (wzheng15) & Miaochan Zhou (mz103)

GitHub: https://github.com/MiaochanZhou/IS507-Team-Project-Team-2

1. Motivation

Food insecurity remains a serious problem in many communities, while retailers may discard unsold food because demand is difficult to predict accurately. This contrast motivated us to study retail sales forecasting. If stores can better estimate future demand, they may make more informed stocking decisions, reduce unnecessary surplus, and avoid stockouts. Although our project does not directly measure food waste or food insecurity, accurate demand forecasting is an important first step toward a more efficient and responsible food retail system.

2.Problem

2.1 Prediction Goal, Predictors, Outcome, and Evaluation

Our goal is to predict the daily unit sales of food items in four California stores for the next 28 days. The unit of prediction is an item-store-day combination.We focus on one state because sales patterns may differ across states due to regional demand, economic and demographic conditions, and different Supplemental Nutrition Assistance Program schedules(SNAP). We chose California because it has four stores in the dataset, while Texas and Wisconsin have three stores each. This gives us more stores for comparing model performance while keeping the scope of the project manageable.

The predictors X will include historical sales features, such as sales from 1, 7, 14, 28, and 56 days earlier; rolling sales averages over the previous 7-56 days; item prices; weekdays; months; holidays and other calendar events; SNAP indicators; and product information such as item and department identifiers.

The outcome Y is the number of units of a specific item sold in the store on a given day. During training,our candidate model Light Gradient Boosting Machine (LightGBM), a gradient-boosted decision-tree model, will minimize squared prediction error. We will evaluate and compare LightGBM and the moving-average baseline using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) over the 28 days forecasting period. Lower MAE and RMSE indicate more accurate predictions.

2.2 Data

We will use the dataset from the Kaggle M5 Forecasting—Accuracy competition. The complete dataset contains daily sales for 3,049 products sold in 10 Walmart stores across California, Texas, and Wisconsin. The sales data contain 30,490 item-store series observed over 1,941 days.

For this project, we will use four files: calendar.csv, sales_train_validation.csv, sell_prices.csv and sales_train_evaluation.csv. **calendar.csv** contains dates, weekdays, events, and SNAP information. **sales_train_validation.csv** contains daily unit sales for each product-store combination from day1 to day1913.**sell_prices.csv** contains weekly product prices for each store. **sales_train_evaluation.csv** extends the sales records through day1941.

2.3 Potential impact

1.Reduce overstocking: More accurate forecasts could help stores avoid ordering more products than customers are likely to purchase.
2.Reduce stockouts: Better estimation of demand could improve product availability and reduce lost sales.
3.Reduce food waste: Lower excess inventory may reduce the amount of unsold food that becomes unsuitable for sale.
4.Support earlier redistribution: If stores can identify likely surplus earlier, they may have more time to discount, redistribute, or donate safe food products.

3. Related work

We identified 4 studies relevant to our project.

Makridakis et al. studied the results of the M5 Accuracy competition. They compared different methods for predicting 28 days of Walmart sales and found that LightGBM was used by many of the

top-performing approaches.Unlike the full M5 competition, our project focuses on food products in four California stores and examines whether LightGBM consistently outperforms a simple moving-average baseline.

Ke et al. introduced LightGBM, an efficient gradient-boosted decision-tree algorithm. It builds a sequence of decision trees, with each new tree attempting to correct prediction errors made by the earlier trees. This study helps explain why LightGBM is suitable for the large M5 retail dataset.We build on this work by using LightGBM as our candidate model and testing whether it produces more accurate forecasts than a simple moving-average baseline on our selected retail dataset.

Januschowski et al. looked at why tree-based models like LightGBM were used by almost all the top teams in the M5 competition. They found that these models work well because they can naturally handle things like product categories, missing data, and lots of zero sales, all of which also appear in our Walmart data. Building on their findings, we use LightGBM on a smaller subset of the M5 data with similar features like past sales and calendar information. We also go a step further by checking how well the model performs at different levels (individual items, departments, and overall) and comparing it with a simple moving-average baseline.

Deng et al. applied LightGBM to Walmart sales forecasting and compared it with alternative machine-learning methods. They reported lower prediction error for LightGBM, providing another practical example of using gradient boosting for retail sales prediction. We follow a similar approach but compare LightGBM with a simple moving-average baseline rather than other ML methods, and further evaluate performance at different levels, from individual items to departments and to the store level, to understand how accuracy varies across different product groupings.

4. Data

4.1 Data Structure and Sources

We will use the official dataset from the Kaggle M5 Forecasting - Accuracy competition (Link: https://www.kaggle.com/competitions/m5-forecasting-accuracy/data).

The dataset contains 4 files.

1) calendar.csv

In calendar.csv, each row represents a single, unique calendar day across the continuous 1,969-day observation period (day 1 to day 1969). Each column captures specific temporal, event, and policy attributes for that date:

date: Records the calendar date in YYYY/MM/DD format;
wm_yr_wk: Specifies the Walmart year-week identifier used to link with sell_prices.csv;
weekday: Indicates the full name of the day of the week (Monday to Sunday);
wday: Records the numeric index of the day of the week, ranging from 1 to 7 (where 1 represents Saturday and 7 represents Friday in Walmart's calendar system);
month: Specifies the numerical month of the year (1 to 12);
year: Specifies the calendar year (e.g., 2011, 2012);
d: Identifies the sales day, ranging from d_1 to d_1969, and is used to connect the calendar data with the sales data;
event_name_1: Specifies the name of the primary event or holiday occurring on that date;
event_type_1: Categorizes the primary event into one of four types: Cultural, National, Religious, or Sporting;
event_name_2: Specifies the name of a secondary event or holiday occurring on the same date;
event_type_2: Categorizes the secondary event into its corresponding type (Cultural, National, Religious, or Sporting);
snap_CA: Binary indicator (1 or 0) signifying whether Supplemental Nutrition Assistance Program (SNAP) food subsidies were distributed on that day in California;
snap_TX: Binary indicator (1 or 0) signifying whether SNAP subsidies were distributed on that day in Texas; and

snap_WI: Binary indicator (1 or 0) signifying whether SNAP subsidies were distributed on that day in Wisconsin.

2) sales_train_validation.csv
The dataset contains 30,490 rows, where each individual row represents a unique time series corresponding to a specific item-store combination.

The columns are structured into two main parts:
First 6 columns: These include id (the unique time series identifier), item_id (the product ID), dept_id (the department category), cat_id (the broad category, such as Hobbies, Foods, or Household), store_id (the specific store location), and state_id (the state where the store is located, namely CA, TX, or WI).

Columns from d_1 to d_1913: Contains the historical daily unit sales data per product and store from day 1 to day 1913.

3) sell_prices.csv
The sell_prices.csv file contains information about the price of the products sold per store and date. It has 4 columns and 6,841,122 rows (including the first row).

Each row represents the price of a specific product at a particular store during a given Walmart week. The four columns are: store_id (identifying the store location), item_id (identifying the unique product), wm_yr_wk (a numeric code representing the specific Walmart calendar week), and sell_price (the price at which the item is sold at that store during that week).

4) sales_train_evaluation.csv
The sales_train_evaluation.csv file contains the historical daily unit sales data per product and store. It has 1,947 columns and 30,491 rows (including the first row).

Rows: Each row represents a specific item sold at a specific store. The row records the complete day-by-day sales history of that item at that store.

Columns:
id: a unique identifier combining item_id, store_id, and the suffix "_evaluation";
item_id: the product identifier;
dept_id: the department;
cat_id: the product category;
store_id: the store identifier; and
state_id: the state where the store is located.
The remaining columns (d_1 to d_1941) record the number of units sold on each corresponding day.

The M5 dataset integrates four tables centered on daily sales. sales_train_validation (d_1–d_1913) provides historical data, while sales_train_evaluation extends this to d_1941 and includes the actual sales numbers for the competition's evaluation period (d_1914–d_1941). Both sales tables link to calendar via the d column for temporal features like dates and holidays. Additionally, sell_prices connects to the sales data through wm_yr_wk, store_id, and item_id to supply weekly pricing information for each product-store combination.

4.2 Current Challenges
Several challenges become apparent as we examine the data more closely. First, there are a massive number of zeros in the sales data. Out of a total of 59,181,090 data points, 40,241,819 are zeros. This means that about 68% of our data contains no sales at all. This makes it hard for models like our moving-average baseline and LightGBM to learn meaningful patterns because they have too many "no-sale" examples.

Second, building a complete analysis dataset requires joining multiple tables, specifically sales_train_evaluation.csv, sell_prices.csv, and calendar.csv, using item_id, store_id, and wm_yr_wk as keys. These multiple joins increase computational complexity and processing time.

Third, the files are very large, especially since sell_prices.csv alone has over 6.8 million rows and the sales table has nearly 2,000 daily columns. If we load everything at once, we may run out of memory, so we will need to read the files in smaller parts, for example by processing them in chunks or loading only the columns we need.

Finally, the dataset contains a mix of data types including text labels such as item_id, categorical variables like state_id and cat_id, dates, prices, and integer counts, each of which requires different preprocessing steps before the data can be fed into a model.

4.3 Expected Challenges

Several challenges are expected as we move into the modeling and evaluation stage. First, we plan to check our forecasts at different levels for each single item, for each food department, and for all food items together. When we add up the item-level predictions to get a department total or a grand total, the numbers should make sense and add up correctly. Making sure everything stays consistent across these different levels adds extra work for our evaluation and model tuning.

Second, adding outside information like price changes, holidays, and special events into the model is not easy. Prices can change at any time and do not follow a simple pattern. Holidays and events can cause sales to jump up or drop down suddenly, which is hard to predict. Making these into useful information that both our moving-average baseline and LightGBM can actually use will take careful data preparation.

Third, some items have very little sales history, or none at all. This means the model does not have enough examples to learn from. Items that only show up toward the end of the dataset are even harder because there is even less information available. This lack of data for certain items may hurt the performance of both our baseline and LightGBM, and we may need to deal with these items separately or look at similar products to fill in the gaps.

Finally, because we are splitting the data by time rather than randomly, our training, validation, and test sets must follow the actual order of time. This limits how we can adjust model settings and check how well the model works, since we cannot shuffle the data to get a smoother result. We will need to pick our validation and test time periods carefully so that our evaluation matches how forecasting works in real life.

5. Plan

5.1 Models Under Consideration

We plan to compare two forecasting methods. Our baseline is a historical moving average with a 28-day look-back window. For each item-store combination, it uses the average daily sales during the most recent 28 days to predict daily sales for the following 28 days. We selected this baseline because it is simple, transparent, and consistent with our 28-day forecasting horizon.

Our candidate model is LightGBM, a gradient-boosted decision-tree model. LightGBM builds a sequence of decision trees, with each new tree attempting to correct the prediction errors made by the previous trees. Unlike the baseline, LightGBM can use multiple predictors, including lagged sales, rolling sales averages, prices, weekdays, holidays, SNAP indicators, and product information. We selected LightGBM because it can efficiently process a large retail dataset and capture nonlinear relationships between these predictors and future sales.

We will compare the two methods using the same dataset, temporal splits, 28-day forecasting horizon, and evaluation metrics. LightGBM will only be considered an improvement if it consistently produces lower MAE and RMSE than the moving-average baseline.

5.2 Model Training and Evaluation Plan

We will conduct the analysis in Python using a Jupyter Notebook in Visual Studio Code.We plan to focus on food products sold in the four California stores. This scope allows us to compare forecasting performance across stores while keeping the computation more manageable than using all stores and states.Using pandas, we will reshape the sales data into a long format in which each row represents the daily sales of one item. We will then merge the sales data with the price and calendar datasets.

For each item and date, the predictors will include sales from 1, 7, 14, 28, and 56 days earlier; rolling sales averages over the previous 7, 28, and 56 days; price; weekday; month; holidays; SNAP status; and product information. The outcome variable will be the number of units sold on that date.

We will use a temporal split rather than a random split. Earlier observations will be used for training, followed by a 28 days validation period and a final 28 days test period. Days day1 through day1885 will form the training period, day 1886 through day 1913 will form the 28 day validation period, and day1914 through day1941 will form the 28-day test period.We will train LightGBM on the training data and use the validation period to select the number of trees, learning rate, and number of leaves. After selecting these settings, we will retrain the model using the training and validation periods and evaluate it on the untouched test period.

We will generate the 28 days forecast sequentially. Predictions from earlier forecast days will be used to update the lagged and rolling features required for later forecast days. Finally, we will compare LightGBM with the 28 days moving-average baseline using the same product subset, forecast periods, MAE, and RMSE.

5.3 A baseline.

The baseline is a 28 days historical moving average. For each item-store combination, we will calculate the average daily sales during the 28 days immediately before the forecast period. This average will be used as the predicted daily sales for each of the following 28 days.

We selected this baseline for three reasons. First, it is simple and transparent, so its predictions are easy to understand and reproduce. Second, the 28 days period reflects recent demand while covering four complete weekly cycles, which helps smooth short-term daily variation. Third, the baseline uses only historical sales and does not use price, holiday, SNAP, or product information. It therefore provides a clear benchmark for evaluating whether the additional features and complexity of LightGBM produce a meaningful improvement. Both methods will be evaluated on the same forecast periods using MAE and RMSE.

Reference List

Makridakis, Spyros, et al. “M5 Accuracy Competition: Results, Findings, and Conclusions.” *International Journal of Forecasting*, vol. 38, no. 4, Jan. 2022, https://doi.org/10.1016/j.ijforecast.2021.11.013.

Ke, Guolin, et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” *Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017, https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html. Accessed 11 Oct. 2026.

Januschowski, Tim, et al. “Forecasting With Trees.” *Special Issue: M5 Competition*, vol. 38, no. 4, Oct. 2022, pp. 1473–81, https://doi.org/10.1016/j.ijforecast.2021.10.004. Accessed 11 Oct. 2026.

Deng, Tingyan, et al. “Sales Forecasting Based on LightGBM.” *2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE)*, IEEE, 2021, pp. 383–86, https://doi.org/10.1109/iccece51280.2021.9342445. Accessed 11 Oct. 2026.